SMARTCELL – Week 2 Activity Report

Image Preprocessing, Segmentation, Contour Detection and Evaluation

Project: SMARTCELL – AI-Assisted Blood Cell Screening
Team Role: Member 2 – Image Processing and Cell Detection Research
Week: 2

1. Objective

The objective of Week 2 was to begin the practical implementation of the SMARTCELL image-processing pipeline using Python and OpenCV. The work focused on preparing blood-cell images, converting them into binary masks, identifying potential cell regions, and evaluating the detected regions for further development.

2. Activities Completed

Day 1: Dataset Testing

- Created and executed a Python test script to verify that the dataset images could be accessed and loaded successfully.
- Checked the image files available in the Malaria and Acute Lymphoblastic Leukemia (ALL) dataset folders.
- Established the dataset paths required for subsequent image-processing tasks.

Day 2: Image Preprocessing

- Implemented preprocessing.py using Python and OpenCV.
- Resized input images to a consistent resolution of 512 × 512 pixels.
- Converted images into grayscale to simplify further processing.
- Applied Gaussian Blur to reduce noise.
- Used Contrast Limited Adaptive Histogram Equalization (CLAHE) to improve local image contrast.
- Saved the processed images in separate output folders for Malaria and ALL.

Day 3: Image Segmentation

- Implemented segmentation.py to separate foreground regions from the background.
- Applied Otsu thresholding to automatically determine a binary threshold.
- Applied adaptive thresholding to handle local variations in image intensity.

- Used morphological opening and closing to reduce small noise regions and improve foreground structures.
- Saved segmentation outputs separately for both datasets.

Day 4: Contour Detection and Candidate Region Extraction

- Implemented contour_detection.py using OpenCV contour detection.
- Extracted contours from the segmented images.
- Applied area-based filtering to remove regions outside the selected size range.
- Generated bounding boxes around candidate regions.
- Extracted candidate image crops for further inspection.
- Saved annotated images, candidate crops, and a CSV file containing region information.

Day 5: Candidate Evaluation

- Implemented day5_evaluation.py to evaluate the detected candidate regions.
- Compared normal and inverted Otsu threshold masks.
- Applied filtering using area, minimum width and height, aspect ratio, and circularity.
- Selected the mask polarity that produced more valid candidates under the implemented filtering rules.
- Generated annotated images showing candidate bounding boxes.
- Recorded candidate details in evaluation_report.csv, including image name, threshold polarity, candidate ID, area, coordinates, width, height, aspect ratio, and circularity.

3. Technologies and Tools Used

- **Programming Language:** Python
- **Image Processing Library:** OpenCV (cv2)
- **Numerical and Image Handling:** NumPy, where required by the environment
- **Data Reporting:** Python CSV module
- **Visualization:** Matplotlib
- **Development Environment:** Visual Studio Code
- **Datasets:** Malaria blood-cell images and ALL blood-cell images

4. Project Folder Structure

```
member2/
├── dataset/
│   ├── malaria/
│   └── all/
├── results/
│   └── week2/
│       ├── malaria/
│       ├── all/
│       ├── segmentation/
│       │   ├── malaria/
│       │   └── all/
│       ├── contours/
│       │   ├── malaria/
│       │   └── all/
│       └── evaluation/
│           ├── malaria/
│           └── all/
└── src/
    ├── day1_test.py
    ├── preprocessing.py
    ├── segmentation.py
    ├── contour_detection.py
    └── day5_evaluation.py
```

5. Results and Observations

- Successfully tested dataset image loading.
- Implemented and executed the preprocessing pipeline for both datasets.

- Generated binary segmentation masks using multiple thresholding techniques.
- Detected contours and extracted candidate regions from the processed images.
- Generated annotated outputs and CSV evaluation reports.
- Observed that candidate detection performance varies between Malaria and ALL images. Some Malaria images contain incorrectly positioned or unnecessary bounding boxes, indicating that the current filtering method requires further improvement.

The detected regions are **image-processing candidates only**. They do not confirm the presence of an infected cell or establish a medical diagnosis.

6. Limitations Identified

- Threshold-based segmentation is sensitive to image contrast, background variation, staining, and cell overlap.
- Contour detection can identify unwanted regions in addition to relevant cells.
- Selecting a threshold polarity based only on the number of candidates does not guarantee better biological accuracy.
- The current implementation does not provide a trained AI classification model or clinically validated screening results.

7. Plan for Week 3

- Improve bounding-box accuracy for Malaria images.
- Investigate false-positive and missed candidate regions.
- Test alternative preprocessing and segmentation parameters.
- Compare detected candidate regions against available dataset annotations, where available.
- Prepare cleaner visual outputs and evaluation results.
- Continue toward a future cell-classification pipeline.

8. Conclusion

During Week 2, the initial image-processing pipeline for SMARTCELL was implemented using Python and OpenCV. The work progressed from dataset validation to image preprocessing, segmentation, contour detection, candidate extraction, and basic evaluation. Outputs were generated for both Malaria and ALL datasets. The results provide a foundation for further refinement of cell detection and subsequent AI-based

classification. The next phase will focus on improving detection quality and evaluating the results more systematically.